

第26回 エラーを読んで直す

今回のゴール

- エラーを「コンパイルエラー」「実行時エラー(例外)」「論理エラー」の 3 種類に分けられる。
- エラーメッセージと行番号から、原因の候補を探して修正できる。
- `try-catch` で、数字以外の入力でプログラムが止まらないようにするコードを読める。

前回とのつながり

第25回までで、バトルのプログラムは 7 つのクラスになった。コードが長くなるほど、書き間違いやバグも見つけにくくなる。第17回から「数字を入力する」と約束していた入力の問題も残っている。今回は、壊れたバトルのプログラムを、エラーを読みながら直していく。

実習時のフォルダ構成

```
ワークスペース
├─ java26_sample
│   └─ src
│       └─ battle
│           └─ (第25回の解答例と同じ 7 ファイル。Main.java に try-catch を追加)
└─ java26_exercise
    └─ src
        └─ battle
            └─ (バグが 7 か所入った 7 ファイル)
```

- 配布ファイル: `プログラム/第26回/sample/battle/`、`プログラム/第26回/exercise/battle/`
- 解答例(直した後): `プログラム/第26回/answer/battle/`

講義内容

1. エラーの 3 つの種類

種類	いつ分かるか	例	見つけ方
コンパイルエラー	保存したとき・実行する前	<code>;</code> がない、型が合わない、名前の書き間違い	Pleiades の赤い印とメッセージ
実行時エラー(例外)	実行している途中	範囲外の添字、 <code>null</code> のまま使う、数字以外の入力	コンソールの赤い文字(<code>Exception in thread "main" ...</code>)

種類	いつ分かるか	例	見つけ方
論理エラー	実行結果を見たとき	計算式・条件の間違い	予想と実行結果を比べる。エラーは表示されない

2. 実行時エラーの読み方

```
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index 3 out of bounds for length 3
    at java.base/java.util.ArrayList.get(ArrayList.java:428)
    at battle.Main.main(Main.java:56)
```

- 1 行目: 例外の種類とメッセージ。「長さ 3 のリストで、3 番を使おうとした」。
2. `at ...` の行を上から見て、自分が書いたクラス (`battle.` で始まる行) を探す。`Main.java:56` なら、`Main.java` の 56 行目。
3. その行で、何の値がおかしいのか考える。

よく出る例外	意味
<code>ArrayIndexOutOfBoundsException</code> / <code>IndexOutOfBoundsException</code>	配列・リストにない番号を使った
<code>NullPointerException</code>	<code>null</code> (何も入っていない変数) のメソッドを呼んだ
<code>InputMismatchException</code>	<code>nextInt()</code> で整数以外が入力された
<code>ArithmeticException</code>	0 で割った

3. try-catch: 例外が起きたときの処理を書く

プログラム/第26回/sample/battle/Main.java (変わったところ)

```
import java.util.InputMismatchException;
```

```
static int inputCommand(Scanner scanner, String[] commands) {
    for (int i = 0; i < commands.length; i++) {
        System.out.println((i + 1) + ": " + commands[i]);
    }
    int command = 0;
    do {
        System.out.print("コマンドを選ぶ > ");
        try {
            command = scanner.nextInt();
        } catch (InputMismatchException e) {
            // 数字以外が入力されたときは、ここに来る
            System.out.println("数字を入力してください");
            scanner.next(); // 読めなかった入力を捨てる
        }
    } while (command < 1 || command > commands.length);
    return command;
}
```

- `try { }` の中で例外が起きると、そこで中断して `catch { }` の中に移る。例外が起きなければ、`catch` の中は実行されない。
- `catch (InputMismatchException e)` は、「この種類の例外が起きたとき」という意味。
- `scanner.next();` で、読めなかった文字(例: `a`)を捨てないと、次の `nextInt()` でも同じ文字を読もうとして、くり返し失敗する。
- `command` の最初の値を `0` にしておくことで、失敗したときは `while` の条件で聞き直しになる。
- `try-catch` は、この授業では「読んで意味が分かる」ことを目標にする。自分で書けなくてもよい。

4. 実行結果

1 回目の入力で `a` を入力する。

```

コマンドを選ぶ > a
数字を入力してください
コマンドを選ぶ > 1
勇者の攻撃！ スライムに8のダメージ
魔法使いは火の魔法をとなえた！ スライムに10のダメージ
戦士のおのの一撃！ スライムに10のダメージ
スライムをたおした！

```

- 第17回のように止まらず、もう一度聞き直している。

5. エラーを直すときの手順

1. いちばん上のエラーから 1 つずつ直す。1 つのまちがいが、ほかのエラーを引き起こしていることがある。
2. 行番号の行だけでなく、その前の行 も見る(`;` の書き忘れは、次の行でエラーになることがある)。
3. 直したら、保存して、もう一度実行する。
4. 論理エラーは、「正しいときはどうなるはずか」を先に決めておき、実行結果と比べて見つける。

演習1 サンプルを実行する

`java26_sample` でサンプルを実行し、数字以外を入力しても止まらないことを確認する。

演習2 壊れたバトルを直す

`java26_exercise` を作り、`battle` パッケージに `プログラム/第26回/exercise/battle/` の 7 ファイルを入れる。このプログラムには、7 か所のまちがい がある。

(配布した状態では、コンパイルエラーがある)

(直した後は、最後まで実行できる)

直したところは、次の表に記録する。

#	ファイル	行	エラーの種類	メッセージ・症状	直し方
1					
2					
...					

演習2-1 コンパイルエラー(3 か所)

- Pleiades で赤い印が付いているファイルを開き、メッセージを読んで直す。
- 3 か所とも、これまでの授業で一度は見たエラー。

ヒント:

- `;` が足りない。
- `double` の計算結果を `int` の変数に入れている(第18回)。
- 変数名の書き間違い。

演習2-2 実行時エラー(2 か所)

- コンパイルエラーがなくなったら実行する。赤い文字の例外が出たら、例外の種類と、`battle.` で始まる行の行番号を読む。
- 1 つ直したら、もう一度実行する。次の例外が出たら、同じように直す。

ヒント:

- `for` 文の条件で、`i` はいくつまで増えるか。リストの最後の番号は `size() - 1`。
- `NullPointerException` は、「`null` の変数のメソッドを呼んだ」という意味。その変数に、どこで何を入れているか探す(Pleiades では、メッセージに変数名が表示される)。

演習2-3 論理エラー(2 か所)

エラーは出ないが、結果がおかしいところがある。1 ターン目に `2` (薬草)、あとは `1` を入力して、次の 2 つを確認する。

- 勇者の HP は、最大 HP(30)を超えないはず。
- 最初の敵 3 体と、4 ターン目に出るスライムをすべてたおすと、経験値の合計は $5 + 12 + 30 + 5 = 52$ のはず。

ヒント:

- HP の上限を決めているのは、どのクラスのどのメソッドか。`Math.max` と `Math.min` のどちらを使うべきか(第25回)。
- 経験値を「足していく」には、`=` と `+=` のどちらを使うべきか。

発展(任意)

- 自分で 1 か所だけまちがいを入れたファイルを作り、となりの人に直してもらう。どの種類のエラーにしたか、相手が当てられるか試す。

確認問題

問1 次のまちがいは、それぞれどの種類のエラーか(コンパイルエラー／実行時エラー／論理エラー)。

- `int x = "3";`
- `int[] a = new int[2];` の後に `a[2] = 1;`
- 回復の処理で、`Math.min` を使うべきところを `Math.max` にした。

問2 出力は何か。

```
try {
    System.out.println("A");
    int[] a = new int[1];
    a[5] = 0;
    System.out.println("B");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("C");
}
System.out.println("D");
```

問3 次のコードを実行すると、どの例外が起きるか。

```
String name = null;
System.out.println(name.length());
```

■ 答え

- 問1: 1. コンパイルエラー(型が合わない) 2. 実行時エラー(添字は 0 と 1 だけ) 3. 論理エラー(エラーは出ないが結果がおかしい)
- 問2: A、C、D (`a[5]` で例外が起きた時点で `catch` へ移るので、B は表示されない)
- 問3: `NullPointerException` (`null` の変数のメソッドを呼んでいる)

振り返り

- 3 種類のエラーを、見つけ方の違いで説明できるか。
- 実行時エラーのメッセージから、自分のコードの行番号を探せたか。
- 論理エラーを見つけるために、何を先に決めておいたか。

次回へのつながり

次回は、ここまでの内容を 1 つにまとめた「完成版のバトル」を読む。攻撃する敵をプレイヤーが選べるようにし、戦闘の記録(ログ)も整えて、ゲームとして一通り動くようにする。